

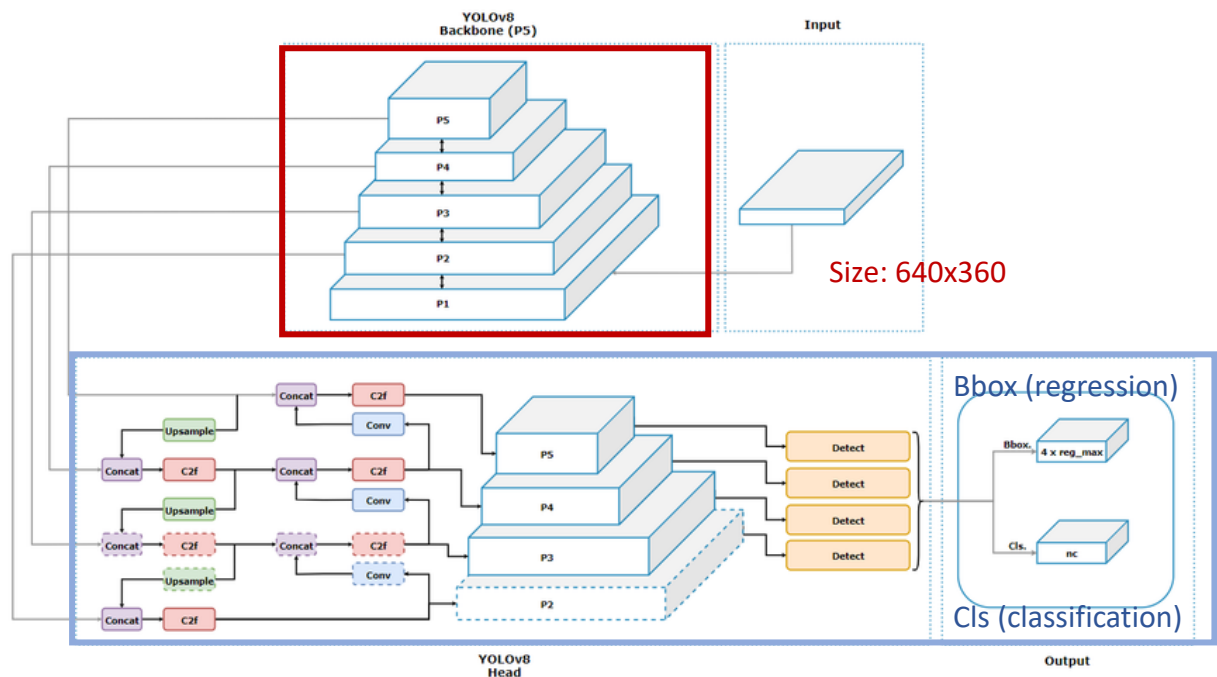
CVPDL hw1 Object Detection Report

School：國立清華大學 National Tsing Hua University

Student ID：114064545

Name：謝尚哲

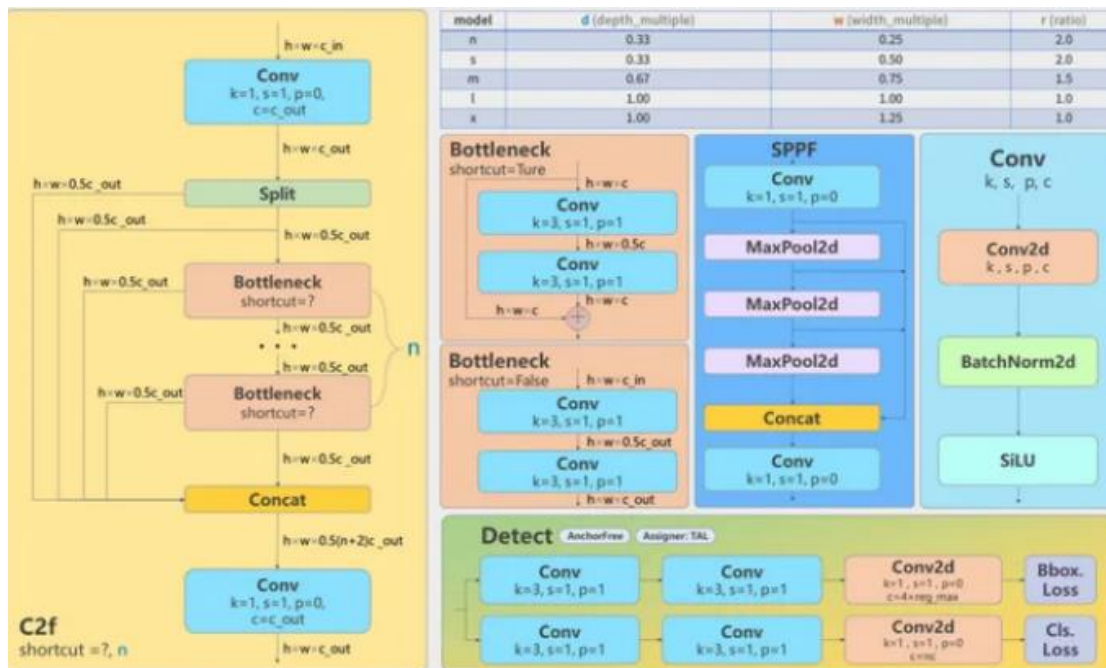
1. Model Description



▲ Fig1. YOLO v8 architecture

The overall architecture of YOLOv8 can be divided into two main parts:

- **Backbone (Red Region)**
 - This part is responsible for feature extraction from the input image. (640*360 for hw1)
 - Based on **CSPDarknet + C2f modules** (In Fig2, , contains a conv block then splitting the output into two halves.) for efficient feature extraction.
 - Instead of the traditional Feature Pyramid Network (FPN) , YOLOv8 includes **SPPF** (In Fig2, Spatial Pyramid Pooling Fast) to aggregate multi-scale information.
 - Outputs three feature maps (P3, P4, P5) with different resolutions.
- **Head (Blue Region)**
 - This part performs **object classification (cls)** and **bounding box regression (bbox)**
 - The head outputs detection results at three scales, matching P3–P5 feature maps.



▲ Fig2. Main Structures of The Main Blocks

2. Implementation Details

• Dataset

For the sake of training with YOLOv8, I have to follow its standard form:

```
dataset/
├── images/
│   ├── train/
│   ├── val/
│   └── test/
├── labels/
│   ├── train/
│   └── val/
└── data.yaml
```

• Data Preprocessing

- All images are **reshaped** to image size **960*960**
- **Data Augmentation** (corresponding to **hyper_pig.yaml**) – horizontal flip (p=0.5), translation $\leq 10\%$, scale $\pm 35\%$, slight rotation ($\pm 1^\circ$), moderate HSV jitter (H=0, S ≤ 0.1 , V ≤ 0.55), and strong early Mosaic (p=0.8) with close-mosaic in the last 40 epochs for stable convergence, plus light MixUp (p=0.1). Vertical flip, shear, perspective, and copy-paste were disabled

due to task semantics.

- Utilizing YOLO loss functions – **bbox loss, cls loss, DFL loss**

- **Training Strategies**

- **Use of a deeper architecture: yolov8x.yaml .**

Training with which has a larger and more complex backbone than others (like yolov8s, yolov8m etc.)

- Training for **sufficient epochs (200 epochs)** in order to converge the loss and get a better performance. Early stopping was also applied (patience=30) to prevent overfitting.
- Utilize the optimizer **AdamW** - Compared with SGD, AdamW provides adaptive learning rate and decoupled weight decay, leading to smoother convergence.
- **Mixed Precision Training (AMP)** - Enabled automatic mixed precision (amp=True) to reduce GPU memory usage and speed up training, without significant loss in numerical stability.
- **Cosine Learning Rate Decay** - for smooth learning rate reduction, promoting steady convergence and preventing overshooting during fine-tuning.
- **Fine-tuning** – After training, I managed to adjust hyperparameters like max detections and confidence/iou threshold, to optimize the F1 score.

3. Result Analysis

- **Quantitative Improvements**

of submission: 33 times

Model	Settings (no use of pretrained weights)	F1-Score
YOLOv11s	Epoch=20, data augmentation, iou=0.55, conf=0.3	0.20336
YOLOv11s	Epoch=200(early stop: patience=30), Optimizer=AdamW data augmentation, iou=0.55, conf=0.3	0.24537
YOLOv11s	Epoch=200(early stop: patience=30), Optimizer=AdamW, data augmentation, iou=0.8, conf=0.0001	0.24917
*WBF (11s / 11m / 11n)	Each model: Epoch=200(early stop: patience=30), Optimizer=AdamW, max_det=150 data augmentation, iou=0.55, conf=0.0001	0.25704
YOLOv11x	Epoch=200(early stop: patience=30),	0.27514

	Optimizer=AdamW, max_det=150 data augmentation, iou=0.7, conf=0.001	
YOLOv11x	Epoch=200(early stop: patience=30), Optimizer=AdamW, max_det=150 data augmentation, iou=0.7, conf=0.001, Using tta during prediction (augment=True)	0.28258
YOLOv8x	Epoch=200(early stop: patience=30), Optimizer=AdamW, max_det=150 -> 180 -> 200 -> 220 (*fine-tuning) data augmentation, iou=0.7, conf=0.001, Using tta during prediction (augment=True)	0.29370 0.29439 0.29485 0.29518



***WBF**: Weighted Box Fusion, merges detection results from multiple models

(YOLOv11s, YOLOv11m, YOLOv11n for example)

*After iterative tuning, **the optimal max_det** value was around **200–220**.

Increasing beyond this point (e.g., 250) provided no further benefit and even risked lowering precision due to excessive low-confidence boxes. Therefore, the final configuration adopts **max_det = 220**, achieving the best trade-off between recall and precision.

• Visualizations

1. Training: Precise bounding box



▲ Fig3. Training batch of YOLOv8x

2. Prediction: Have too many bounding boxes in the image



▲ Fig4. Predict example of YOLOv8x

4. Short Conclusion

Leaderboard Final F1-Score: 0.29518

In this study, we observed two interesting findings.

- **YOLOv8x surprisingly outperformed YOLOv11x** on the dense “swine” dataset, achieving a higher F1-score (0.293 vs. 0.282) despite being an older architecture. With my roughly thinking, I think that YOLOv8’s backbone and feature fusion remain more effective for crowded single-class detection.
- Although setting an **extremely low confidence threshold (=0.001~0.0001)** produced images with plenty of bounding boxes (probably due to the high object density in the test images) the evaluation still **yielded better F1 scores**. This indicates that in ultra-dense scenarios, higher recall outweighs the precision drop, and preserving more candidate boxes can lead to better overall performance.

5. Reference

[1] Detailed Explanation of YOLOv8 Architecture

<https://medium.com/@juanpedro.bc22/detailed-explanation-of-yolov8-architecture-part-1-6da9296b954e>

[2] Image of YOLO v8 architecture

https://www.researchgate.net/figure/Detailed-architecture-of-YOLOv8-showcasing-the-backbone-networks-multiple-convolutional_fig3_385510373

[3] YOLOv8 Architecture: A Deep Dive into its Architecture

<https://yolov8.org/yolov8-architecture/>

[4] YOLOv11 Architecture Explained: Next-Level Object Detection with Enhanced Speed and Accuracy

<https://medium.com/@nikhil-rao-20/yolov11-explained-next-level-object-detection-with-enhanced-speed-and-accuracy-2dbe2d376f71>